PSOC™ Edge E84 training manual: Encryption

About this document

Scope and purpose

This document is a training manual for the hands-on exercise of encrypted boot using Edge Protect Bootloader on PSOC™ Edge using ModusToolbox™.

Intended audience

The intended audiences for this document are design engineers, technicians, and developers of electronic systems.

Table of contents

1 Introduction

This manual provides specific instructions to create, configure, build, and run the code examples on the PSOC™ Edge E84. In this lab session you will learn how to use Edge Protect Bootloader (EPB) on PSOC™ Edge to boot an encrypted application.

2 Required development tools

- ModusToolbox™ software v3.6 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2025.4.0 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.6
- ModusToolbox™ Programming Tools v1.5.0 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.6
- Board support package (BSP)
  - KIT_PSE84_EVAL_EPC2: v1.0.0 or later (available with ModusToolbox™ v3.6)
- Terminal emulator
  - Instructions in this document use Tera Term
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure device boots from external memory
    - BOOT SW/SW6 in 'OFF/LOW’ position
  - Disable alternate serial interface configuration
    - J20/J21 in Tristate/Not-Connected (NC) position

3 Prerequisites

Install the software and obtain hardware listed in the Required development tools section.

This class will not cover basic concepts of ModusToolbox™ and PSOC™ Edge.

- For an introduction to PSOC™, including a getting started guide to ModusToolbox™, visit:

https://www.infineon.com/product-information/psocdeveloper

- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, visit:

PSOC™ Edge E84 Training Collection

4 Single-key XIP encryption with EPB

4.1 Objective

In this lab, we will use the Encrypted Boot code example to demonstrate the XIP encryption capabilities of the Edge Protect Bootloader.

4.2 Description

Firmware images may be encrypted and programmed into the device memory. Edge Protect Bootloader supports the encrypted firmware when EPB runs from RRAM only. Encrypted user images must be located in the external flash memory and executed in SMIF XIP mode or loaded to SRAM for execution.

Edge Protect Bootloader supports the execution of the encrypted application in the following different methods:

- Single-key encryption (EPC 2)
- Multi-key encryption (EPC 2)
- Secure encryption (EPC 4)

This module focuses on using the Encrypted Boot code example for the single-key and multi-key encryption use case for the EPC 2 devices.

4.3 Project creation

1. Before creating the first project we need to create our workspace in ModusToolbox™. Open **ModusToolbox™** from the **Windows Start** menu.

Note: If you have not installed ModusToolbox™, see the Required development tools section.

If you have already created a workspace, please ignore this step and move on to step #4 directly.

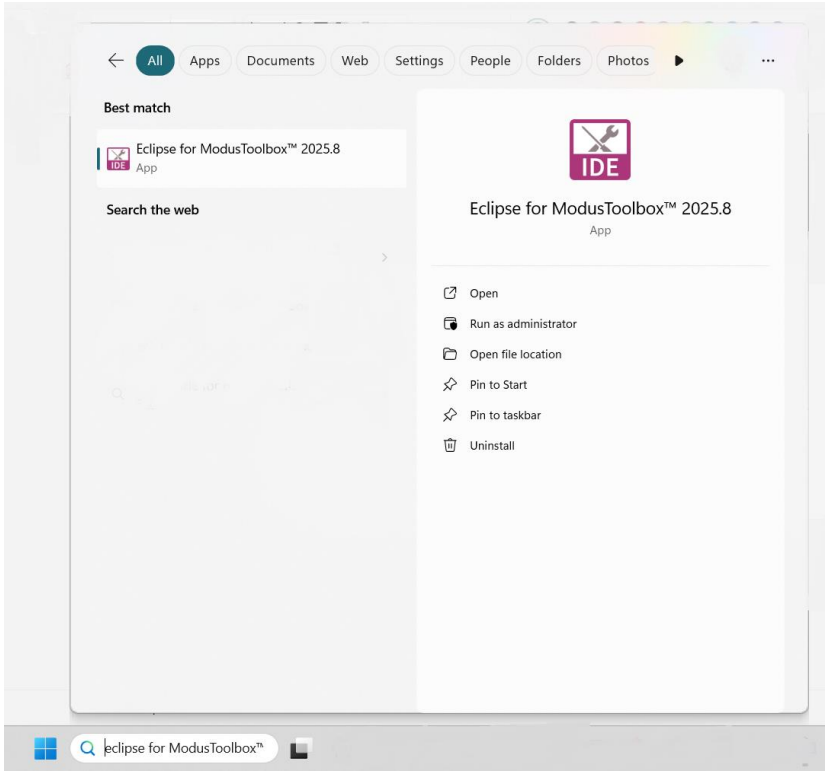


Figure 1 Start ModusToolbox™

2. **Next**, choose a **workspace directory** for your project. After selecting a suitable directory, click **Launch**.

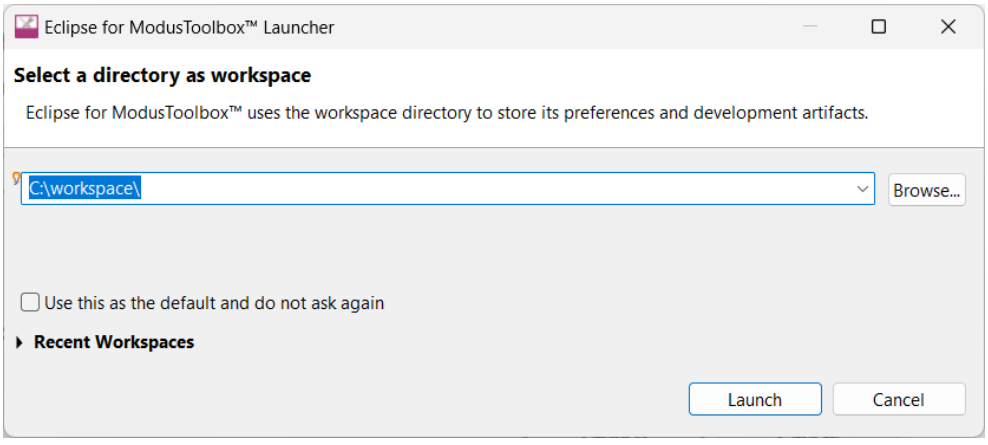


Figure 2 Select workspace directory

3. **Launch** opens a new ModusToolbox™ workspace.

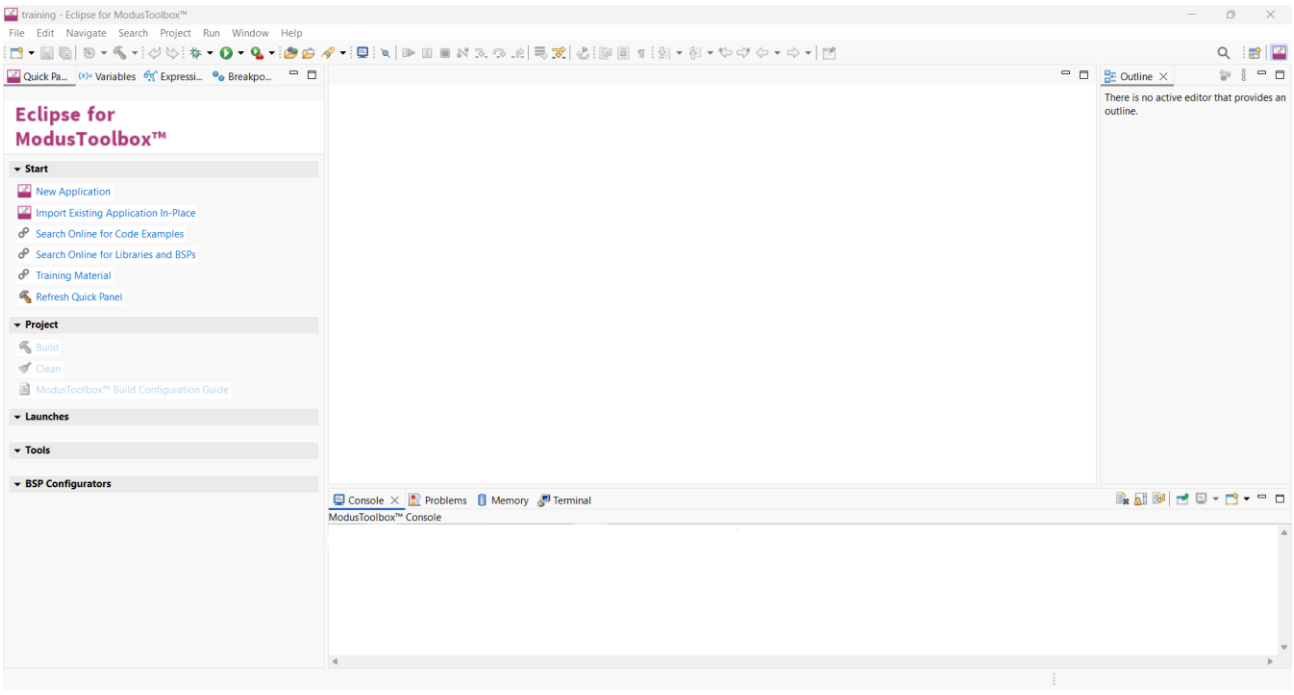


Figure 3 Launch ModusToolbox™ workspace

4. Now select **New Application** from the **Quick Panel**. Alternatively, go to **File** > **New** > **ModusToolbox™** application.

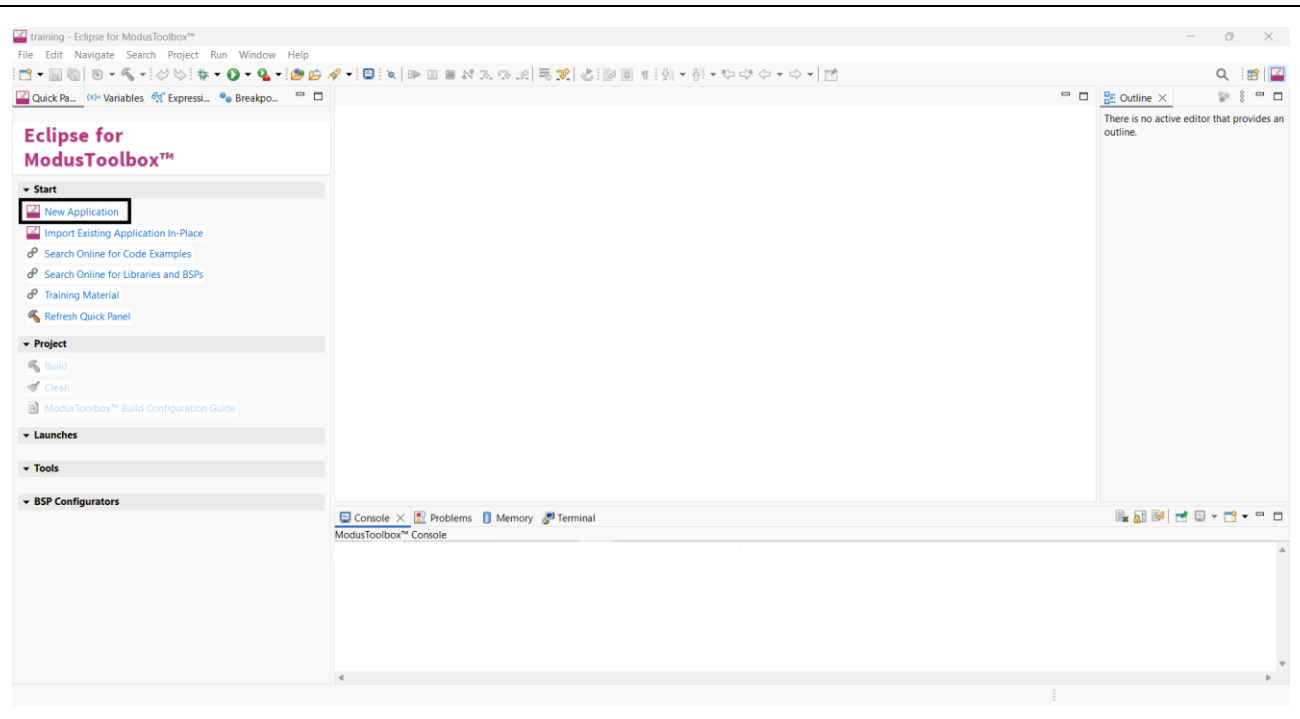


Figure 4 Create new application

5. Clicking New Application opens the **Project Creator** and prompts you to choose a **board support package** (BSP).

*Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*

6. Once **Projector Creator** launches, select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs, and click **Next**.

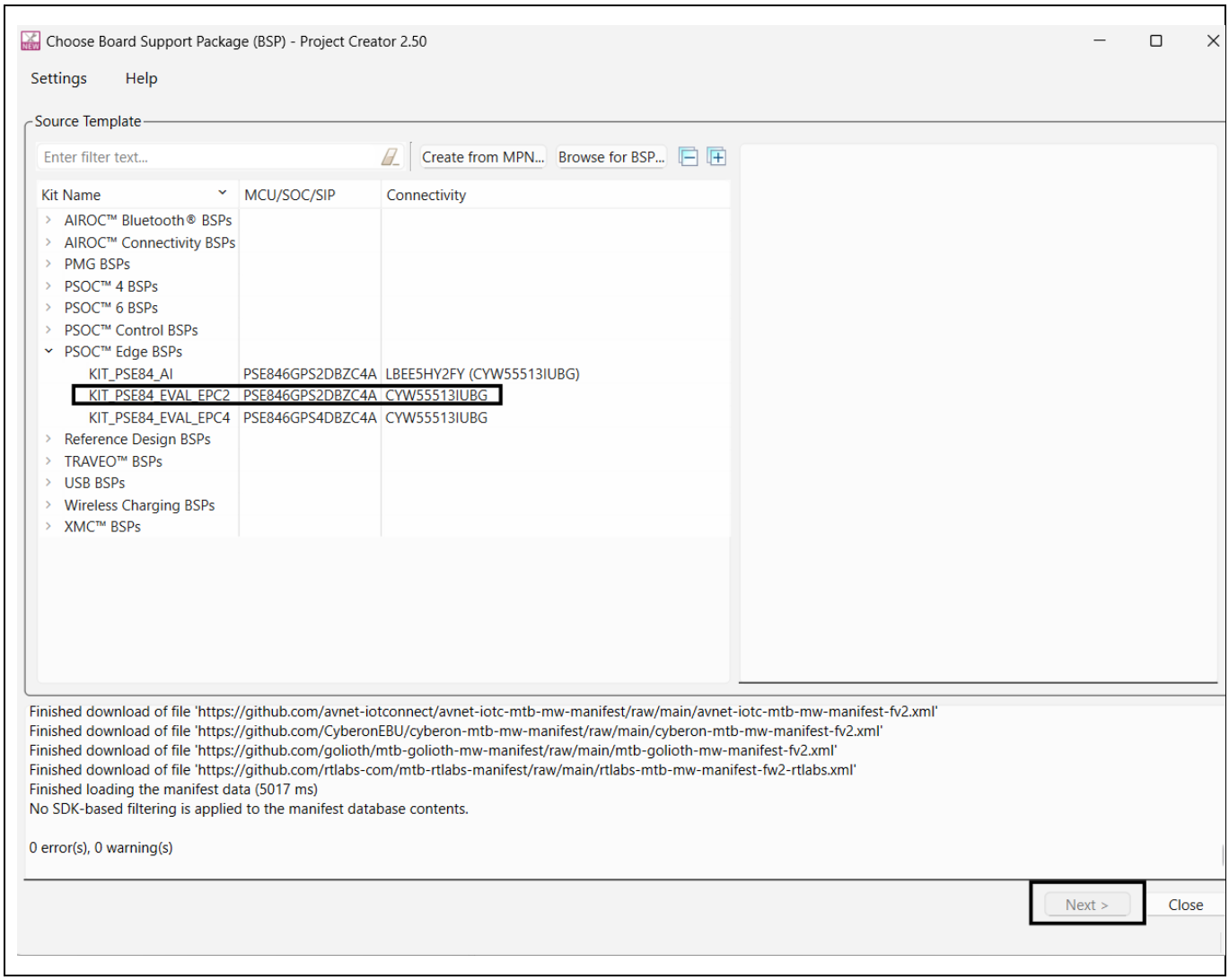


Figure 5 Select BSP in project creator

7. Clicking **Next** takes you to the next menu of the **Project Creator**, which asks you to select an application:
   a) Select the **PSOC™ Edge Protect Bootloader** under the **Security** section. You can do this by selecting the **checkbox** near the application.
   b) Select the **PSOC™ Edge Encrypted Boot** under the **Security** section. You can do this by selecting the **checkbox** near the application.
   c) Click **Create**.

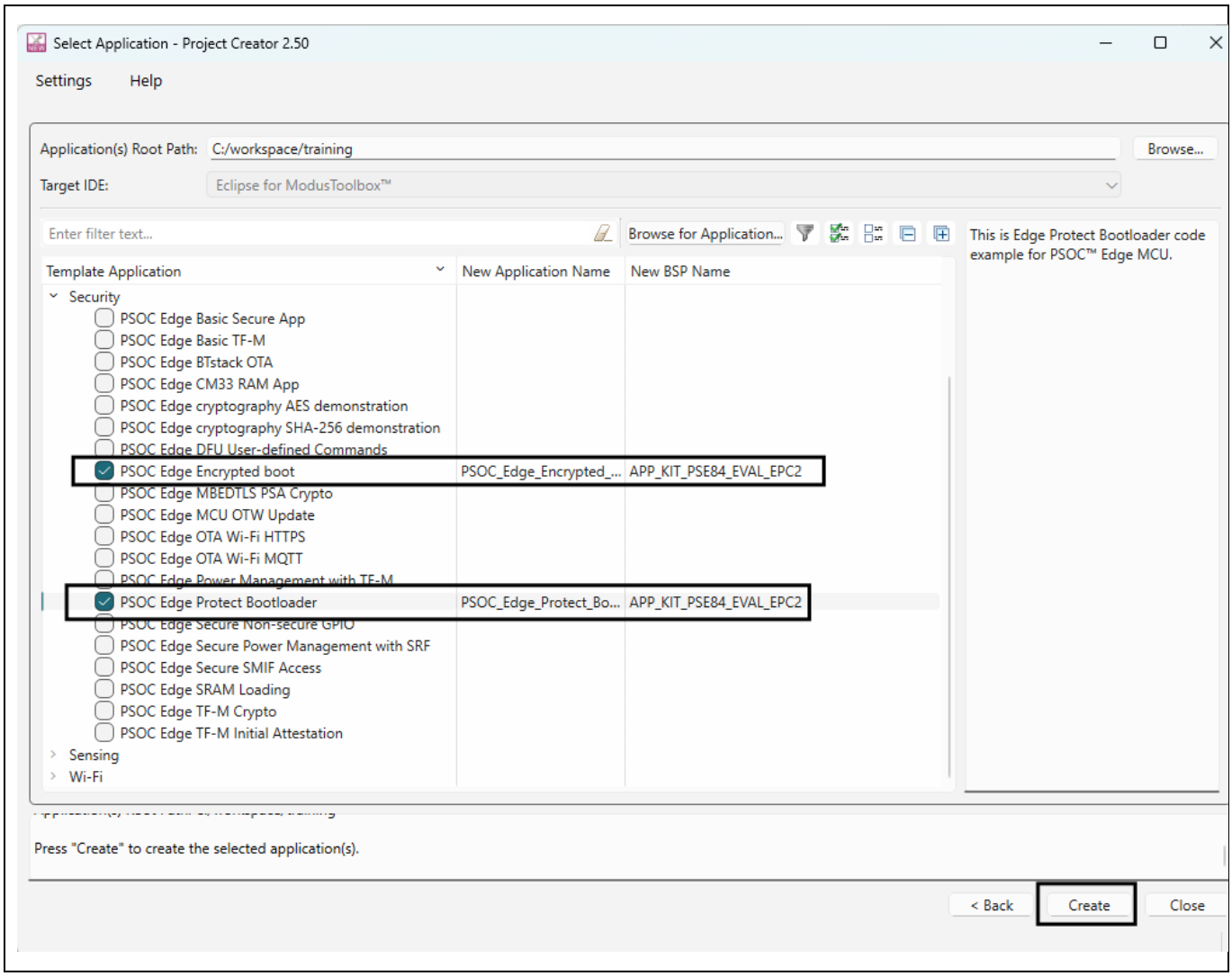


Figure 6 **Select application in project creator**

Note: *Complete demonstration of Edge Protect Bootloader requires Basic Secure App. Creating both code example together in above step.*

8. When you click **Create**, two new ModusToolbox™ applications are created. You can see both applications in **Project Explorer** window.

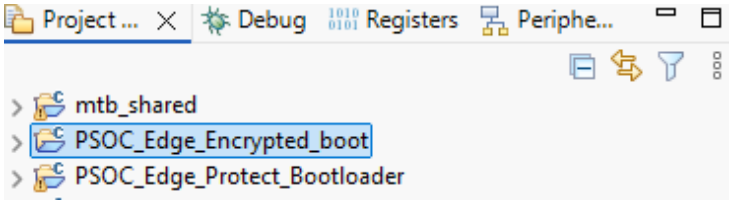


Figure 7 Project explorer window

9. Copy **proj_bootloader** under Edge Protect Bootloader by right clicking on it.

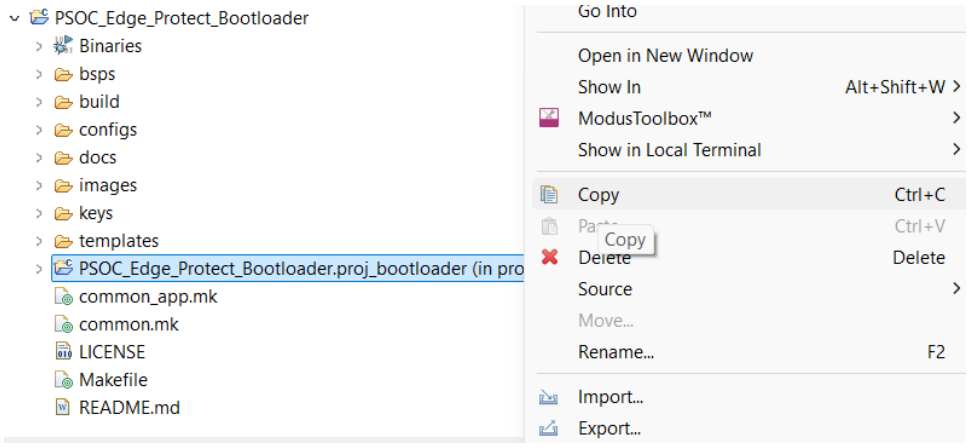


Figure 8 Copy proj_bootloader

10. Right click and paste the **proj_bootloader** to the root of Encrypted Boot.

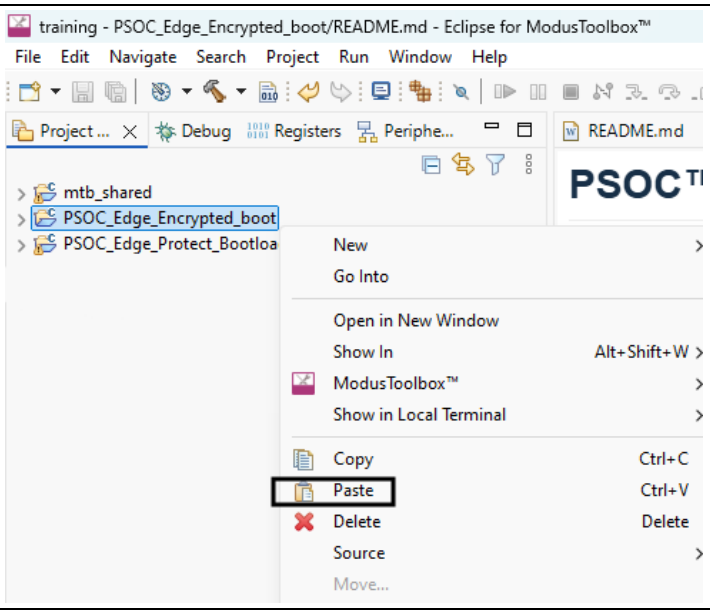


Figure 9 Paste proj_bootloader

11. While pasting edit the project name and location to point to the Encrypted Boot code example.

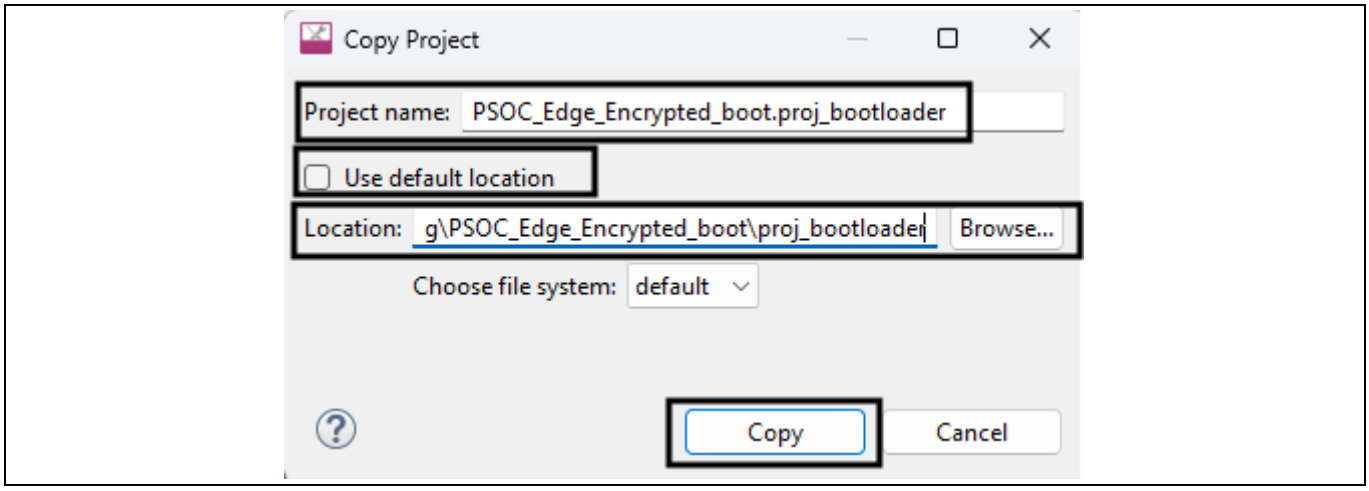


Figure 10 Edit the project name and location

12. Add the **proj_bootloader** to the list of projects in the makefile of Encrypted Boot code example.

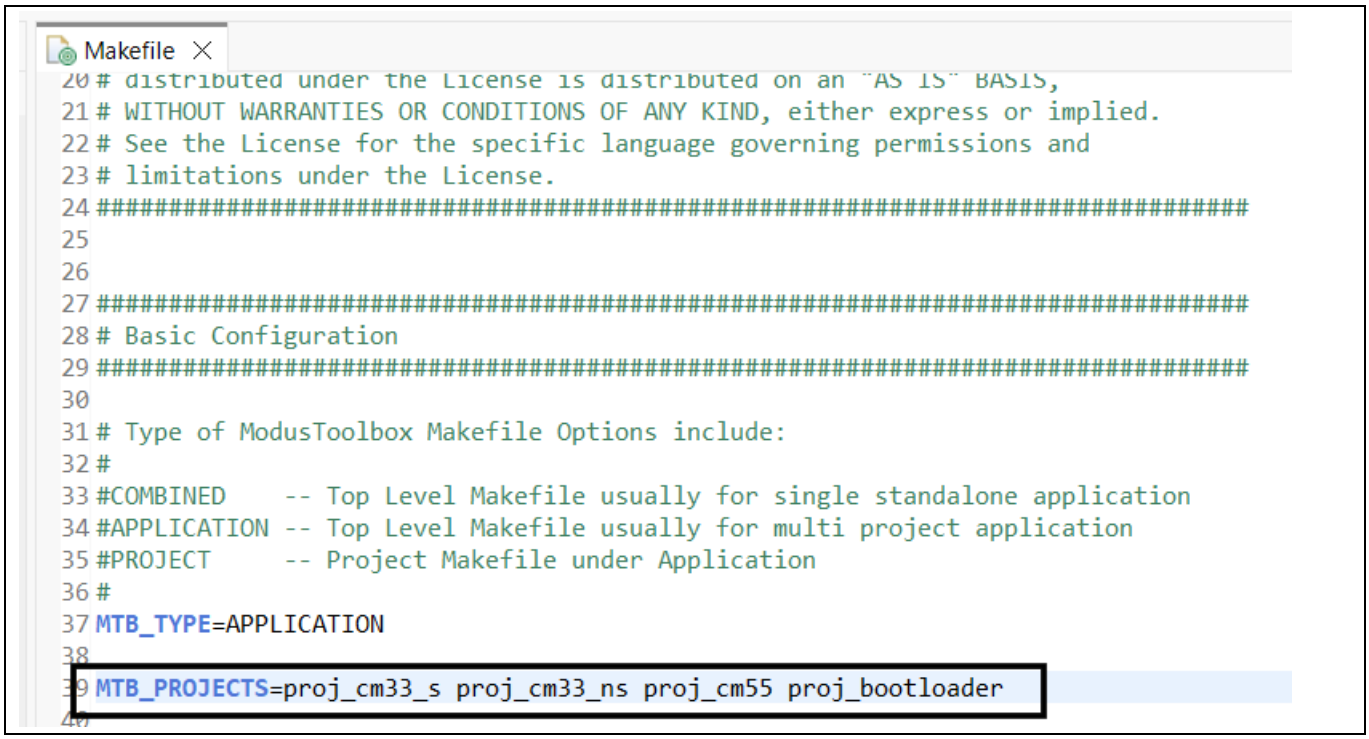


Figure 11 Add proj_bootloader to MTB projects list

13. Update the *common.mk* file of Encrypted Boot to point to bootloader personality generated JSON file.

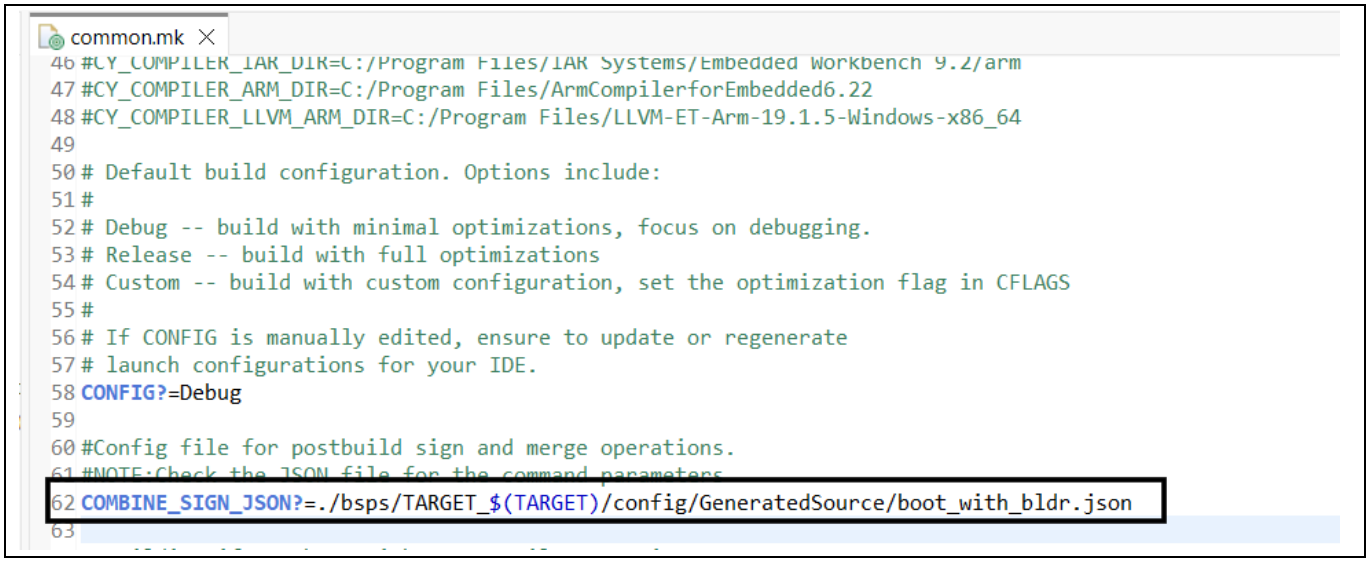


Figure 12 Update *common.mk* file to point to bootloader personality generated JSON file

14. Generate launches for Encrypted Boot code example.

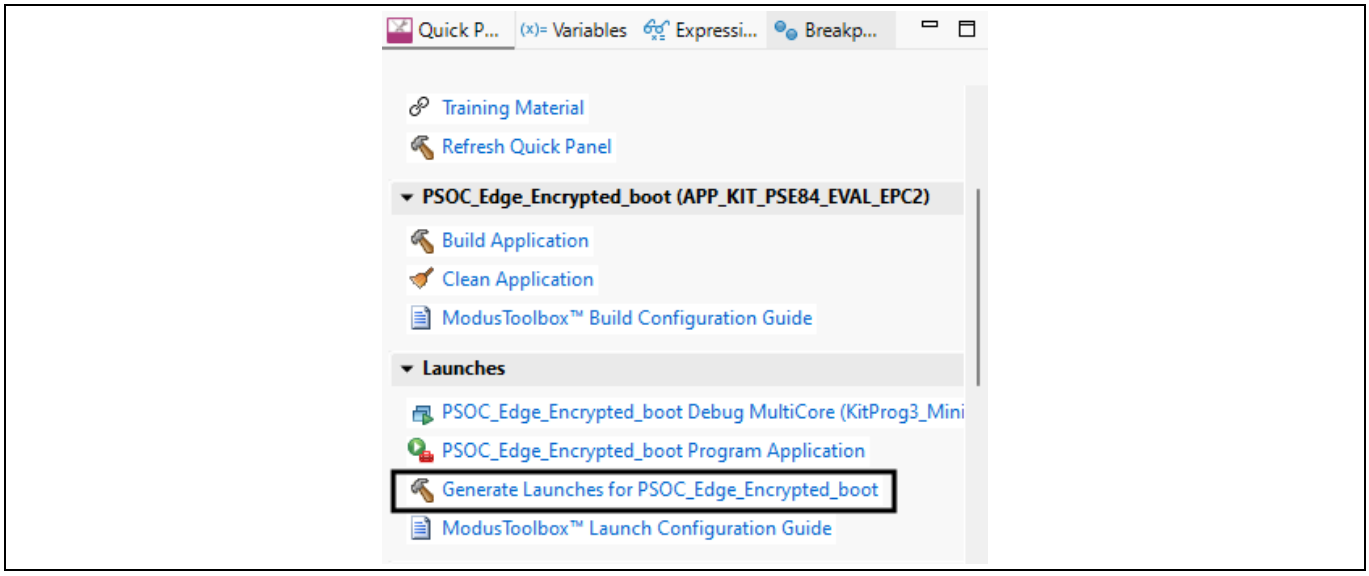


Figure 13 Generate Launches

15. Follow the steps in Edge Protect Bootloader lab module to enable Secure boot flow for the EPB based Encrypted Boot application.

16. Generate the AES-128 Key for image encryption using the following command.

```
edgeprotecttools create-key --key-type AES128 --output keys/aes128_key.bin
```

17. Generate the key encryption key (KEK) using the following command.

```
edgeprotecttools create-key --key-type ECDSA-P256 --output keys/enc-ec256-priv.pem keys/enc-ec256-pub.pem
```

18. Convert the PEM private key to the DER-PKCS8 format using the following command.

```
edgeprotecttools convert-key -k keys/enc-ec256-priv.pem -o keys/enc-ec256-priv.bin -f DER-PKCS8
```

19. Program the private portion of the Key-Encryption-Key (KEK) to the device using openocd at the start address of the “kek_region” defined in the memory map. Check your RRAM memory map in **Device Configurator** for correct location of the **kek_region**. Default configuration places **kek_region** in **0x32039000**.

```
<openocd-path>/openocd.exe -s scripts -f interface/kitprog3.cfg -f target/infineon/pse84xgxs2.cfg -c "init; reset init; flash write_image keys/enc-ec256-priv.bin 0x32039000; reset; shutdown"
```

20. Now build and program the Encrypted Boot application by clicking the **Program Application** button in the **Quick Panel**.

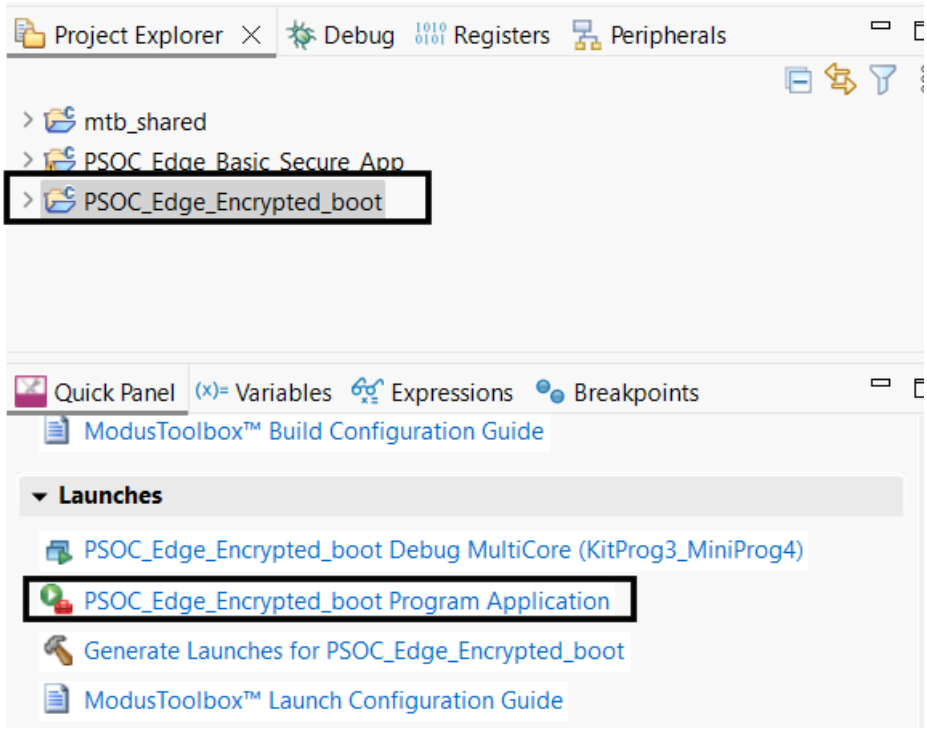


Figure 14 Program application

4.4 Encrypted Boot terminal output

1. Set the Boot SW (SW6)/P17.6 position to “LOW/OFF”. This will instruct Extended boot to boot the first application from RRAM. Edge Protect Bootloader is the first application in this case, which is built and programmed to RRAM in default case.
2. Reset/power cycle the device.
3. Observe the Tera Term console logs for Encrypted Boot.

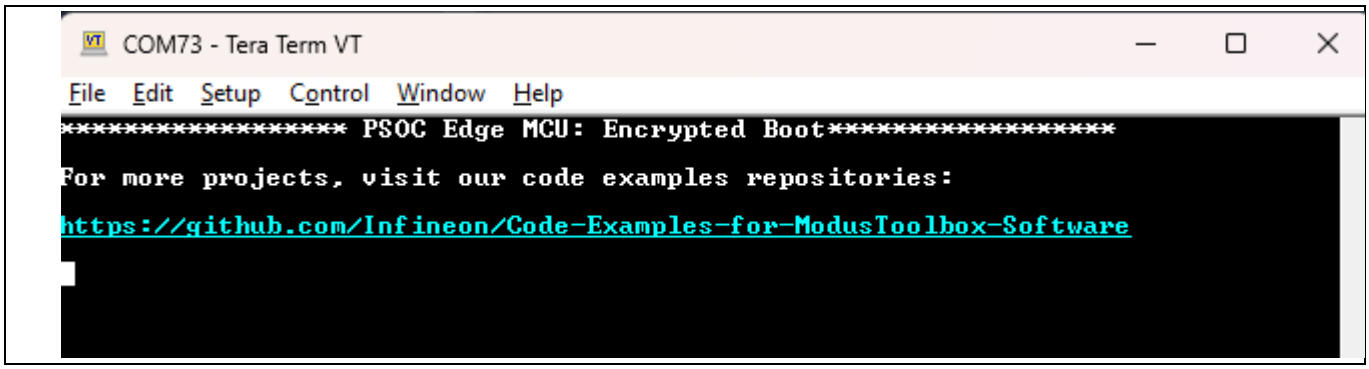


Figure 15 Terminal output for Encrypted Boot

4.5 Conclusion

We successfully tested the single-key encryption using Encrypted Boot code example and learnt:

- How to create the Encrypted Boot code example
- How to add bootloader to the code example and enable the secure boot
- How to program and boot to the encrypted application

5 Multi-key XIP encryption

This is a do-it-yourself exercise.

However, you can see the following steps for the solution.

5.1.1 Project creation

We will use the same Encrypted Boot code example that was created in the previous module for this demonstration.

If not already done, follow the process in Project creation.

5.1.2 Enable multikey encryption

1. If not already done, enable the Secure boot flow for your application.
2. Create 3 AES encryption keys in your application directory.

```
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_1.bin
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_2.bin
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_3.bin
```

3. Generate the key encryption key (KEK) using the following command.

```
edgeprotecttools create-key --key-type ECDSA-P256 --output keys/enc-ec256-
priv.pem keys/enc-ec256-pub.pem
```

4. Convert the PEM private key to the DER-PKCS8 format using the following command.

```
edgeprotecttools convert-key -k keys/enc-ec256-priv.pem -o keys/enc-ec256-
priv.bin -f DER-PKCS8
```

5. Program private portion of the KEK to the device using openocd at the start address of the “kek_region” defined in the memory map. Check your RRAM memory map in *Device Configurator* for correct location of the **kek_region**. Default configuration places **kek_region** in **0x32039000**.

```
<openocd-path>/openocd.exe -s scripts -f interface/kitprog3.cfg -f
target/infineon/pse84xgxs2.cfg -c "init; reset init; flash write_image
keys/enc-ec256-priv.bin 0x32039000; reset; shutdown"
```

6. Open the **Device Configurator** and configure the encryption mode to multi-key encryption in the bootloader solution.

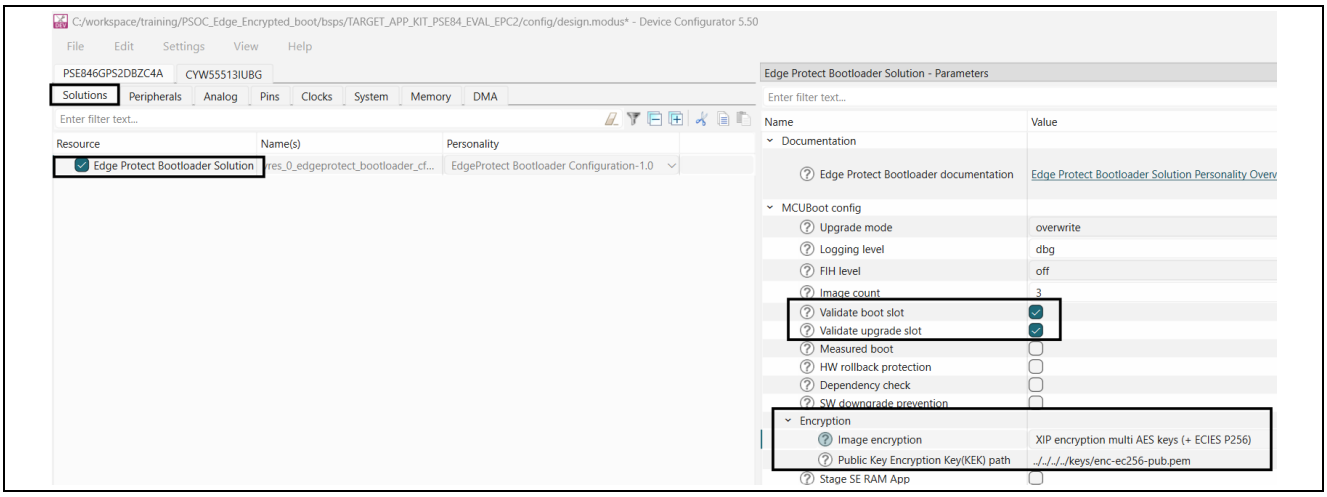


Figure 16 Enable multi-key encryption

7. Set the AES key path for each image for each image.

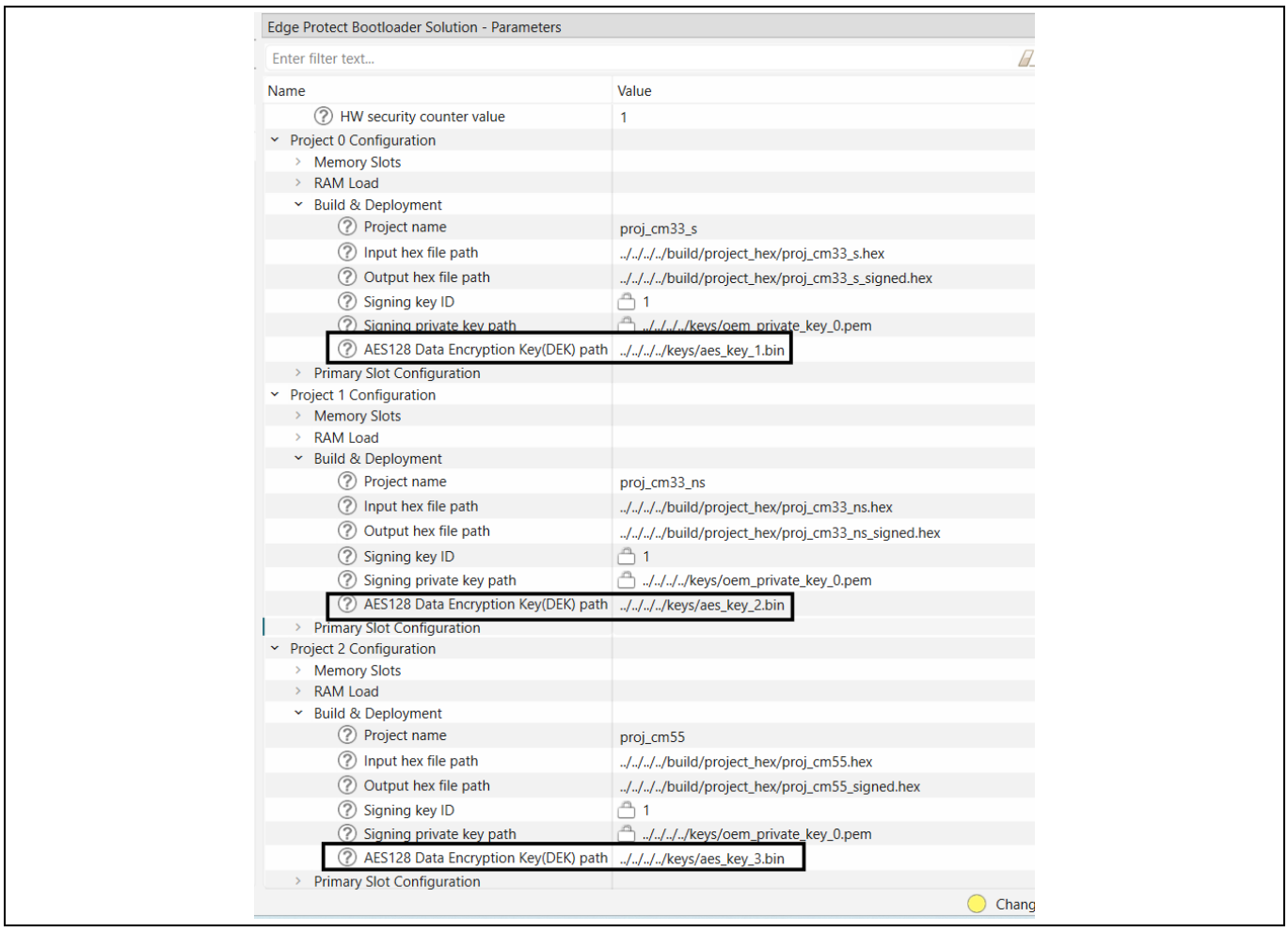
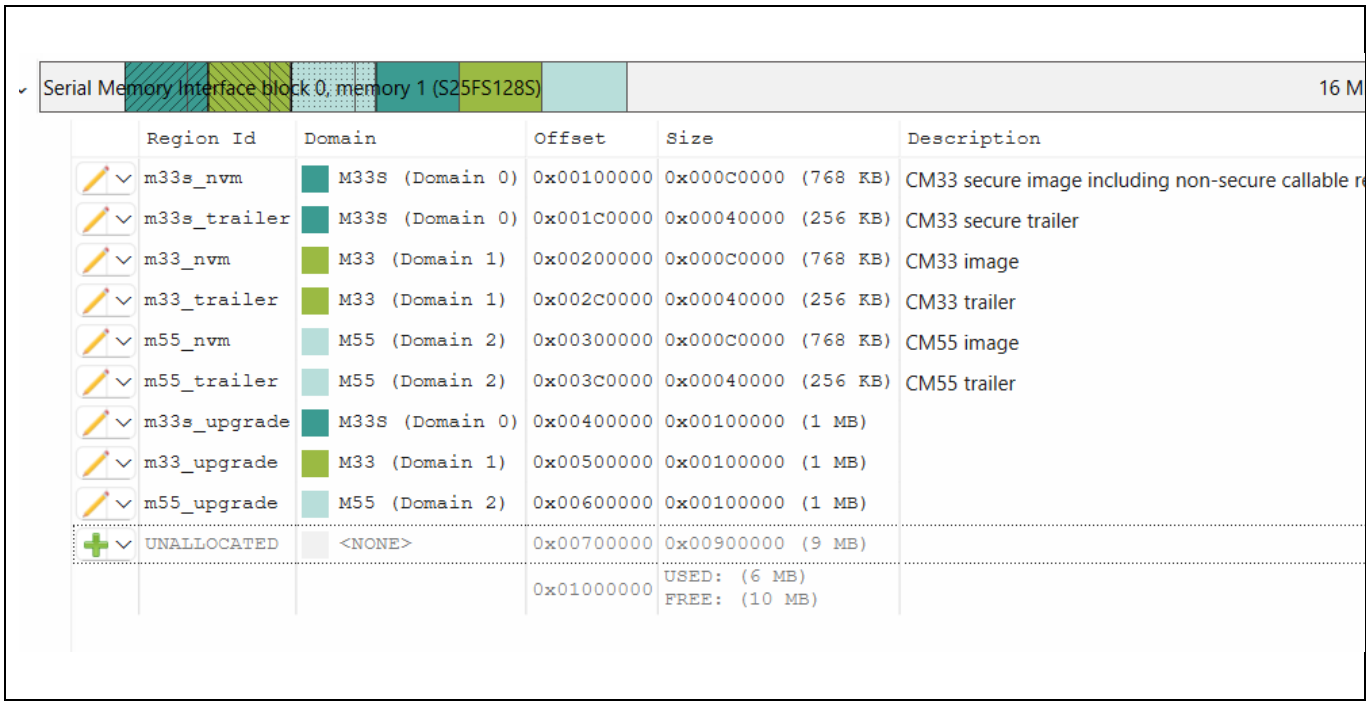


Figure 17 Set AES encryption key

8. Multi-key XIP encryption has strict requirements for encryption region size and start address. The size must be perfect power-of-two number. The start address must be a multiply of size number: start address = N * size; to meet these requirements projects memory regions must be updated. In the example below all images are sized 0x100000.



Region Id	Domain	Offset	Size	Description
m33s_nvm	M33S (Domain 0)	0x00100000	0x000C0000 (768 KB)	CM33 secure image including non-secure callable r
m33s_trailer	M33S (Domain 0)	0x001C0000	0x00040000 (256 KB)	CM33 secure trailer
m33_nvm	M33 (Domain 1)	0x00200000	0x000C0000 (768 KB)	CM33 image
m33_trailer	M33 (Domain 1)	0x002C0000	0x00040000 (256 KB)	CM33 trailer
m55_nvm	M55 (Domain 2)	0x00300000	0x000C0000 (768 KB)	CM55 image
m55_trailer	M55 (Domain 2)	0x003C0000	0x00040000 (256 KB)	CM55 trailer
m33s_upgrade	M33S (Domain 0)	0x00400000	0x00100000 (1 MB)	
m33_upgrade	M33 (Domain 1)	0x00500000	0x00100000 (1 MB)	
m55_upgrade	M55 (Domain 2)	0x00600000	0x00100000 (1 MB)	
UNALLOCATED	<NONE>	0x00700000	0x00900000 (9 MB)	
		0x01000000	USED: (6 MB) FREE: (10 MB)	

Figure 18 Sample memory regions for Multi key encryption

9. Save your changes in the device configurator.
10. Build and program the Encrypted boot application.

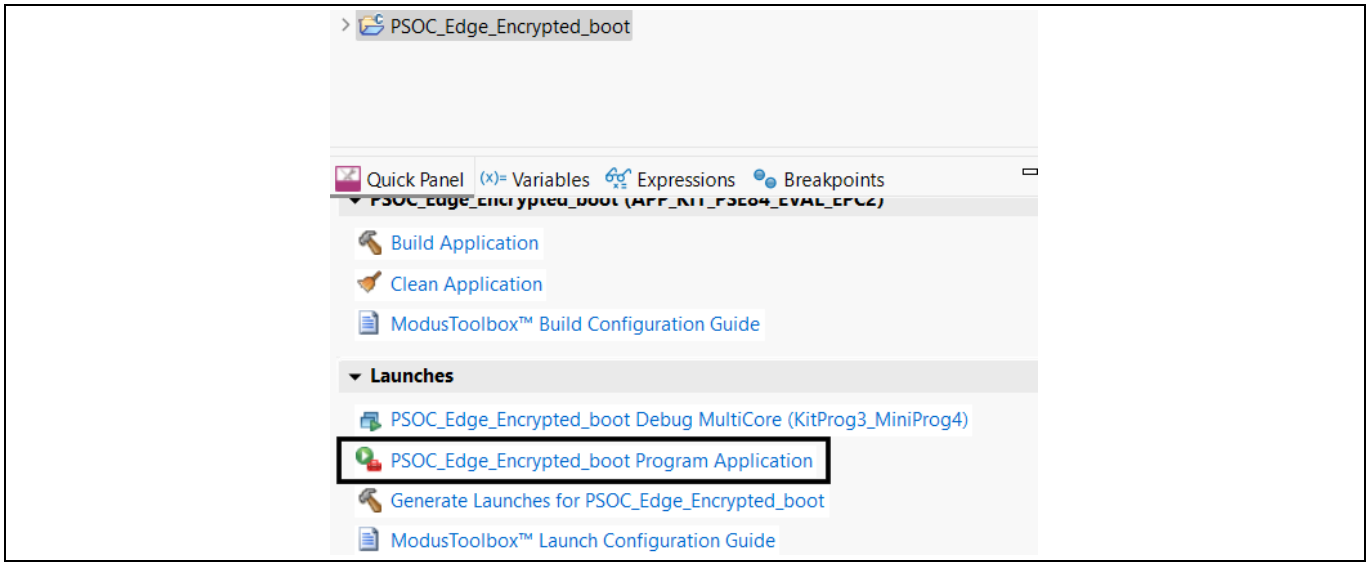


Figure 19 Program application

5.2 Conclusion

We successfully tested the multi-key encryption using Encrypted Boot code example, and learned:

- How to create the Encrypted Boot code example
- How to add bootloader to the code example and enable the secure boot
- How to program and boot to the encrypted application

Revision history

Document revision	Date	Description of changes
**	2026-01-14	Initial release

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition 2026-01-14

Published by

Infineon Technologies AG

81726 Munich, Germany

Do you have a question about this document?

Email:

erratum@infineon.com